

2. CARPETA DE ANÁLISIS

ANEXO 1.A - PRODUCT BACKLOG - ECOLOGISTICS WEB

Historias de Usuario (trazabilidad y criterios de aceptación)

US01 — Reporte de incidencia (Ciudadano) — Must

Historia: Como ciudadano, quiero comunicar una incidencia (contenedor dañado o basura acumulada) adjuntando foto y ubicación, para que el ayuntamiento pueda gestionarla con rapidez. Criterios de aceptación:

Formulario con foto (obligatoria)

Ubicación (mapa o geolocalización)

Tipo y descripción.

Al enviar, se crea incidencia con estado abierta y notificación a administradores.

Consentimiento RGPD requerido.

US02 — Notificación de resolución (Ciudadano) — Should

Historia: Como ciudadano, quiero recibir un aviso cuando la incidencia que he enviado haya sido resuelta.

Criterios: notificación push/email al cambiar estado a resuelta

US03 — Marcar contenedor vaciado (Conductor) — Must

Historia: Como conductor, quiero poder indicar que un contenedor ha sido vaciado, para que el sistema registre su nivel de llenado como 0%.

Criterios:

Botón “Marcar vaciado” en vista conductor; actualiza contenedor.nivel_llenado = 0 y registra timestamp

US04 — Itinerario optimizado (Conductor) — Must

Historia: Como conductor, quiero recibir en mi móvil el itinerario optimizado de la jornada.

Criterios:

Ruta generada para la jornada

Mapa con paradas y orden

Posibilidad de navegación

US05 — Asignación manual de ruta (Administrador) — Should

Historia: Como administrador, quiero asignar manualmente una ruta adicional a un camión.

Criterios:

Interfaz admin para asignar ruta a camion y conductor

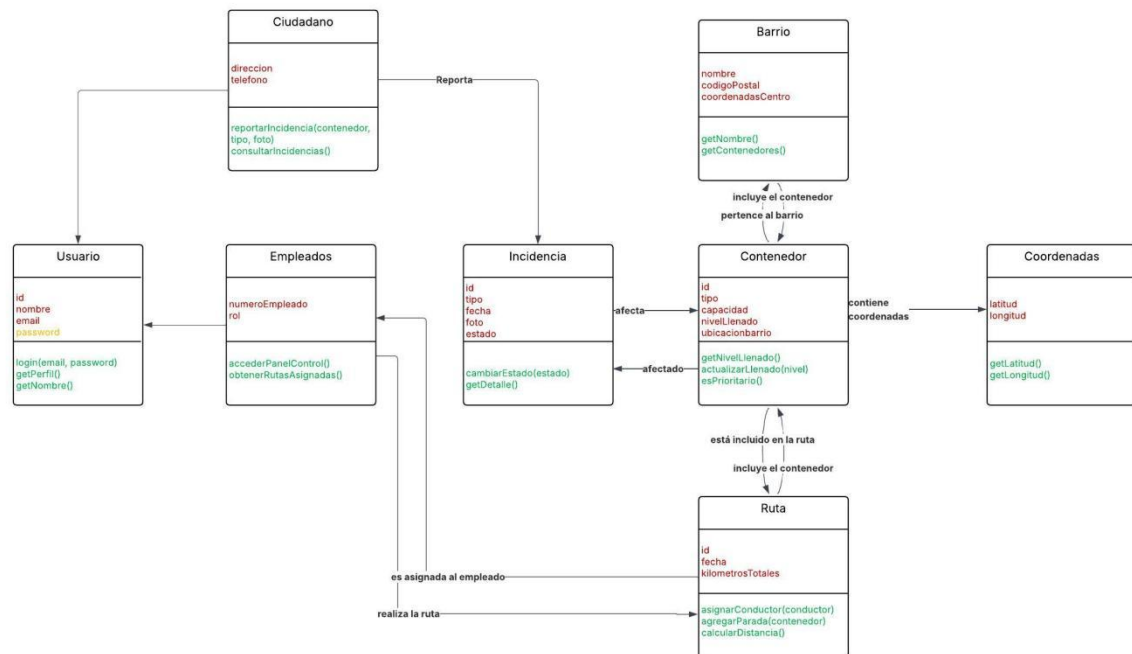
US06 — Gestión de perfiles (Administrador) — Must

Historia: Como administrador, quiero gestionar perfiles de conductores (crear, eliminar, actualizar). Criterios:

CRUD usuarios con rol driver

ANEXO 2.A DIAGRAMAS UML (CLASES)

DIAGRAMA DE CLASES



CLASES PRINCIPALES

Usuario (abstracto)

Atributos: id: UUID; nombre: String; email: String; telefono: String; rol: Enum{citizen, driver, admin}

Métodos: autenticar(), actualizarPerfil()

Ciudadano (Usuario)

Atributos: direccion: String

Métodos: reportarIncidencia()

Conductor (Usuario)

Atributos: licencia: String; camionesAsignados: List<Camion>

Métodos: recibirRuta(), marcarVaciado(contenedorId)

Administrador (Usuario)

Atributos: permisos: Set<String>

Métodos: asignarRuta(), gestionarUsuarios()

Contenedor

Atributos: id: UUID; codigo: String; tipo: String; geom: Point; capacidad_litros: Int; nivel_llenado: Int; estado: String

Métodos: getNivelLlenado(), actualizarNivel()

Camion

Atributos: id: UUID; matricula: String; capacidad_litros: Int; estado: String

Métodos: asignarRuta()

Ruta

Atributos: id: UUID; nombre: String; conductor: Usuario; camion: Camion;

fecha_programada: Date; stops: List<Contenedor> (ordenado)

Métodos: generarItinerario(), optimizarOrden()

Incidencia

Atributos: id: UUID; reportero: Usuario; contenedor: Contenedor; tipo: String; descripcion:

Text; estado: String; foto_url: String; created_at; resuelto_at

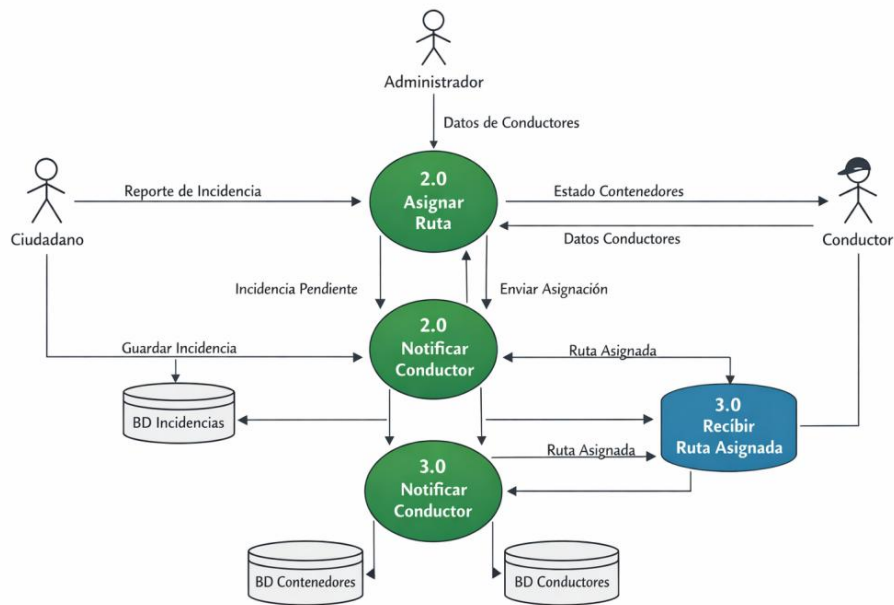
Métodos: cambiarEstado()

Notificacion

Atributos: id: UUID; usuario: Usuario; tipo: String; payload: JSON; leida: Boolean; created_at

Métodos: marcarLeida()

ANEXO 2.B DIAGRAMAS DE FLUJO



Generación de rutas

Flujo: Selección candidatos → Agrupación por zona → Asignación por capacidad → Optimización 2-opt → Asignación camión/conductor → Persistencia

Entradas

Reporte de Incidencia (foto, ubicación, tipo)
Datos de contenedores (nivel de llenado, ubicación)
Datos de conductores (disponibilidad, vehículo asignado)
Parámetros del administrador (prioridad, zona, criterios)

Transformaciones

Validación y registro de Incidencia: según formato, ubicación y se almacena en la BD
Evaluación y priorización: según la gravedad, ubicación y tipo
Asignación de ruta: Incidencias + contenedores + disponibilidad de conductores
Notificación al conductor: sistema envía ruta al dispositivo del conductor

Salidas

Incidencia registrada
Incidencia priorizada
Ruta generada
Notificación enviada al conductor
Confirmación de recepción